

Finding and exploiting/patching attack paths in your Active Directory environment

As an attacker or an analyst during an internal penetration test or a red team assessment, we often ask (ourselves) question like “What can I do with this account I just compromised?” or “How can I quickly move to a highly privileged account from this compromised machine?”.

As a defender, you can ask (yourself) similar questions to find and patch attack paths in your Active Directory environment: “Who can access those valuable targets?” or “What privilege could this external contractor get?”. One cannot easily answer these questions by looking at the Active Directory Users and Computers console, GPOs, etc.

Thankfully, others have asked the same questions before us and three years ago, a few nice guys have come up with a tool that allow asking exactly the questions mentioned above and getting beautiful answers in the form of a graph: Bloodhound.

BloodHound



BloodHound is (according to their Readme

<https://github.com/BloodHoundAD/BloodHound/blob/master/README.md>)

- a single page Javascript web application
- with a Neo4j database
- fed by a PowerShell C# ingestor

BloodHound uses graph theory to reveal the hidden and often unintended relationships within an Active Directory environment.

Getting Started

Using BloodHound is quite simple, and the documentation of the tool is very well done:

1. Grab a precompiled version of the user interface from here
<https://github.com/BloodHoundAD/BloodHound/releases/latest>
2. Install and configure neo4j according to the instructions here
<https://github.com/BloodHoundAD/BloodHound/wiki/Getting-started>
3. Grab a precompiled version of the ingestor from here
<https://github.com/BloodHoundAD/BloodHound/tree/master/Ingestors>
4. Collect data
5. Enjoy.

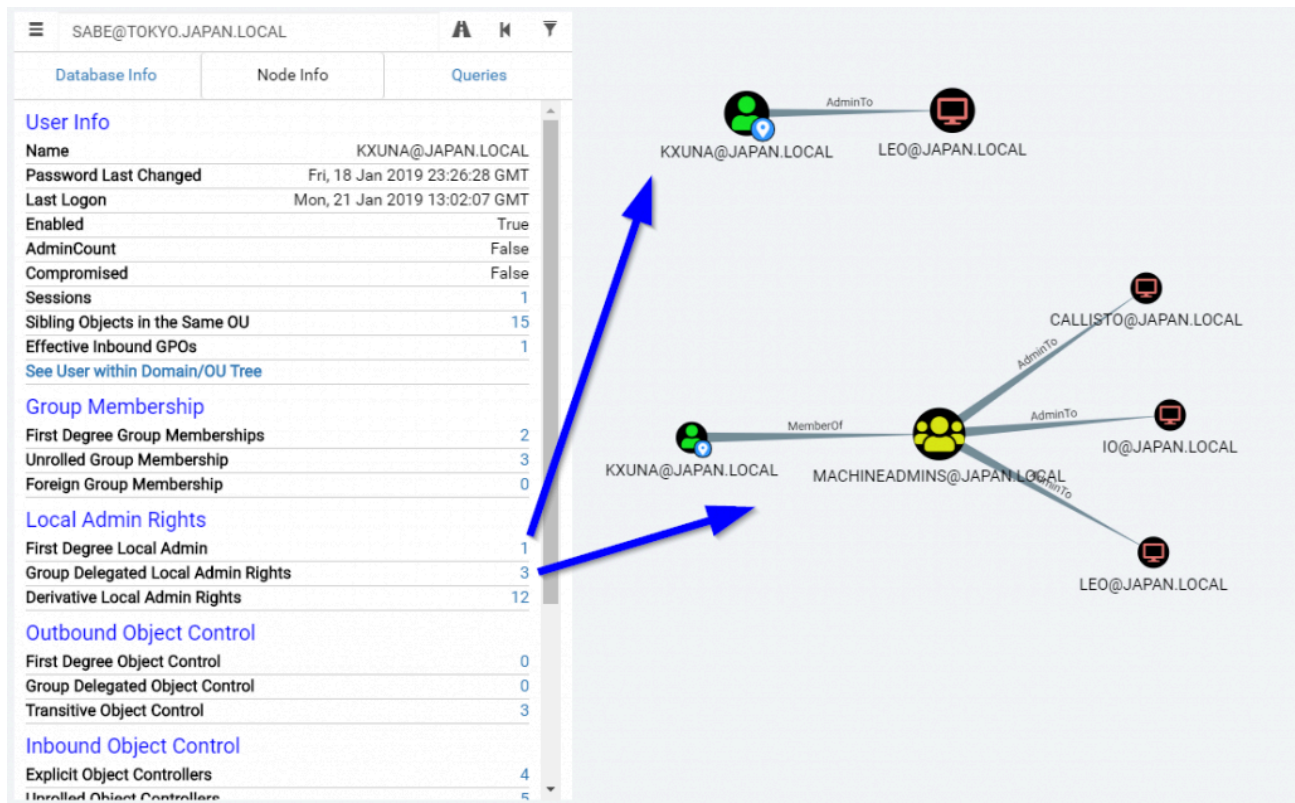
5 useful pieces of information you can get out of BloodHound

1. Local Administrators

(u:User) - [:AdminTo] -> (c:Computer)

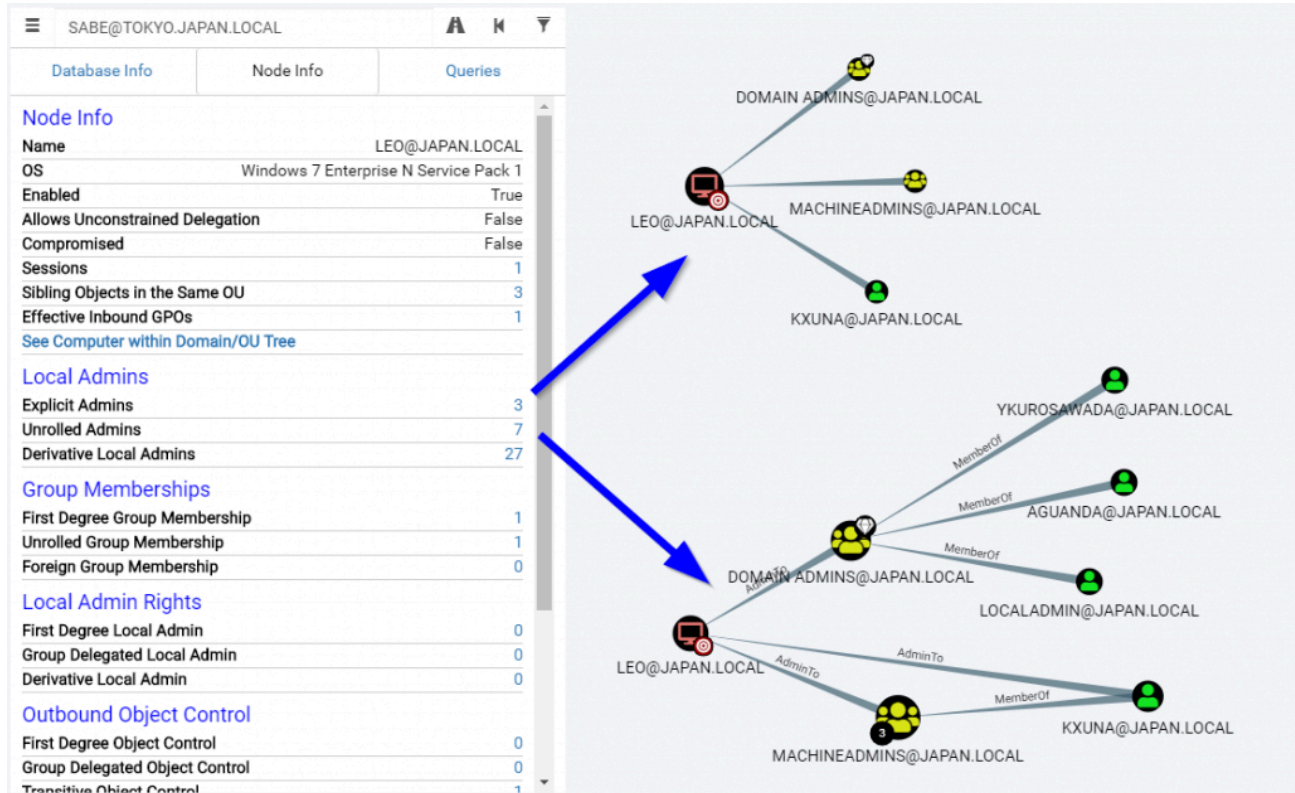
One thing you definitely want to do to tighten your AD security is giving local administrator access to the least people possible. With local administrator rights on a machine, an attacker can have full control, establish persistence, read all data on the disk and in memory, including potentially credentials of other users.

You can simply know on to which machines a user is admin by using the “First Degree Local Admin” and “Group Delegated Local Admin Rights” functions when browsing a user:



User KXUNA has local admin rights on several machines.

The reverse operation is possible, if you want to know which user has administrative rights on a given machine. These functions are called “Explicit Admins” and “Unrolled Admins”:

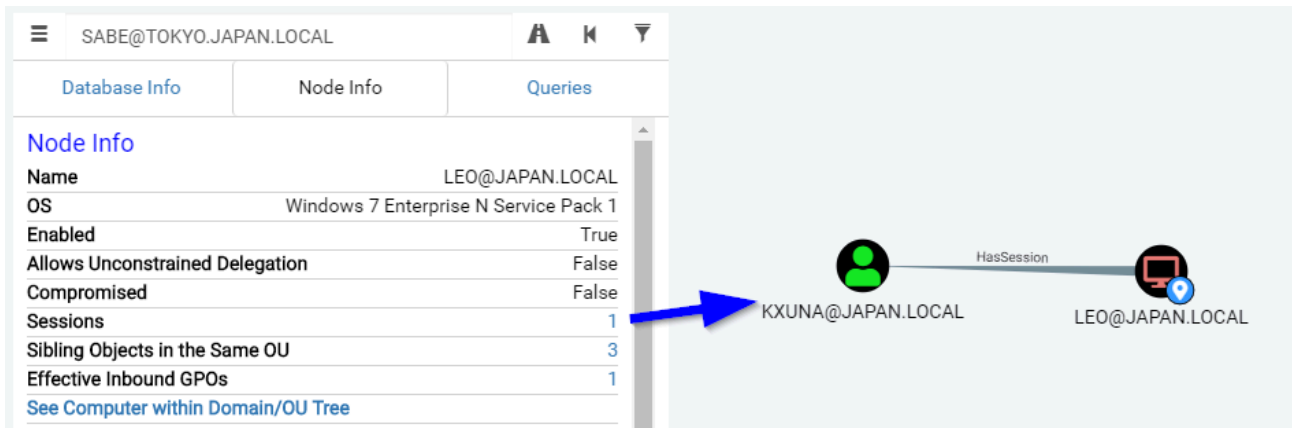


The computer LEO has 7 local admins.

2. Sessions

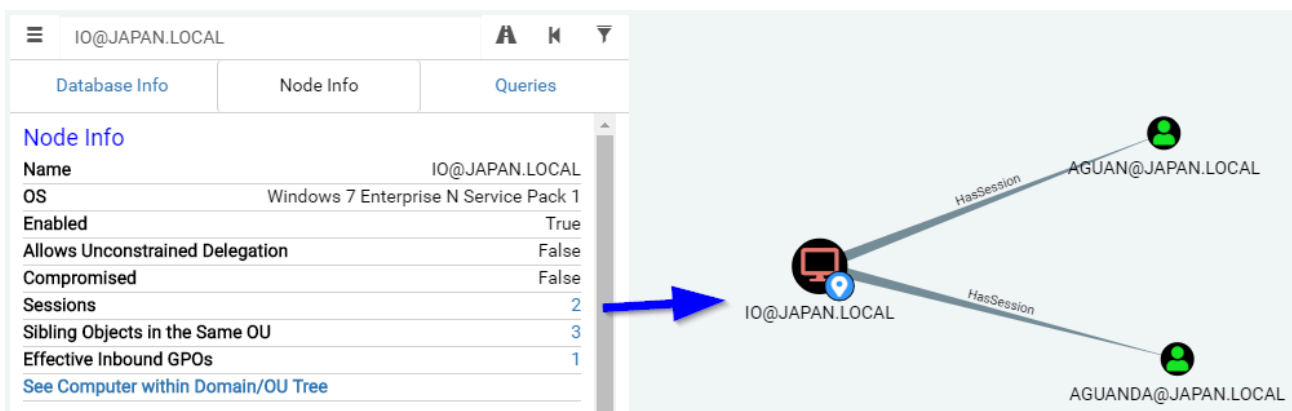
(u:User) - [:HasSession] -> (c:Computer)

As an attacker, our goal is to get privileged access as soon as possible. It is sometimes difficult to identify which machine is worth trying to compromise in order to get to credentials of privileged users. Bloodhound allows to list the current sessions of users, this is convenient in order to know, for instance, if Domain Administrators are connected to a workstation:



User KXUNA has a session on computer LEO.

Conversely, it is interesting to know which user have sessions on a given computer:



Two users have sessions on computer IO.

3. ACLs

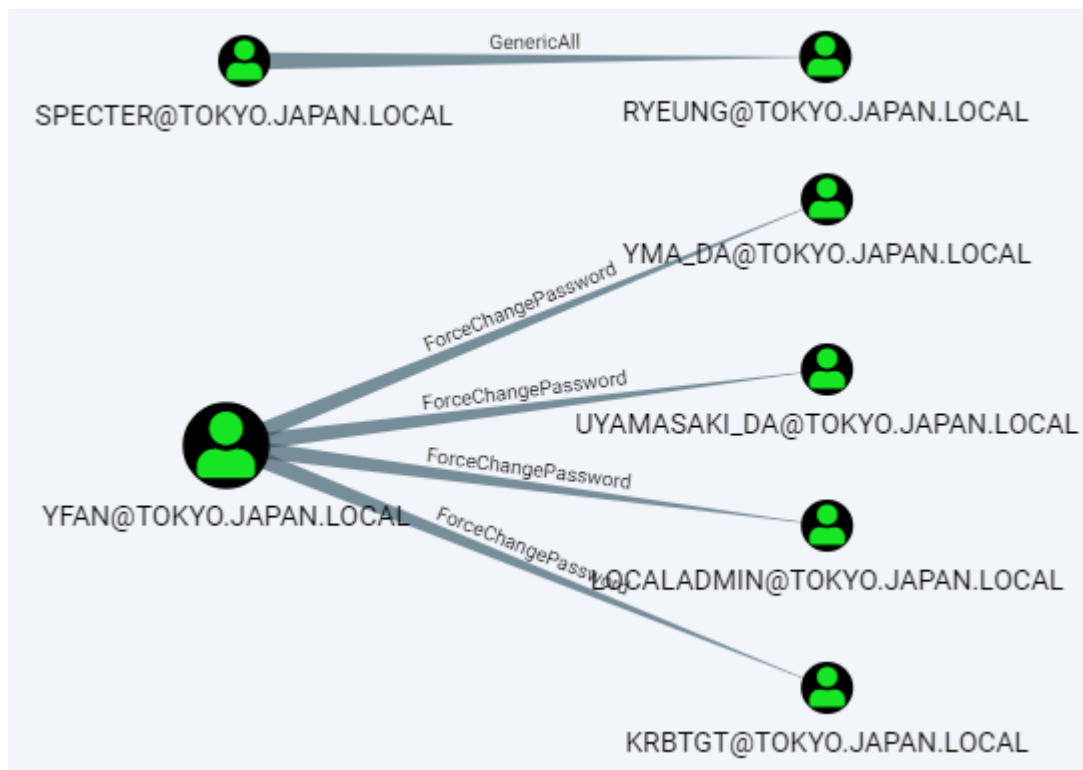
(u:User) - [:GenericAll] -> (g:Group)

Another way to get more privileges in a network is through ACLs. Helpdesk users can often reset user passwords, admin have often full control on many other AD objects, etc. BloodHound allows to track the following ACLs:

- ForceChangePassword,
- AddMembers,
- GenericAll,
- GenericWrite,

- WriteOwner,
- WriteDACL,
- AllExtendedRights,
- Owns,
- ReadLAPSPassword

Example query to display all direct ACL from user to user:



User Specter has GenericAll rights on user RYEUNG.

User YFAN can change passwords of 4 users.

4. Unconstrained Delegation

(c:Computer {UnconstrainedDelegation:true})

Computer allowing unconstrained delegation can be misused by attackers to impersonate almost any user in the domain. As such, these are very sensitive and need to be identified.

The following custom query ([source](#)) lists all non-dc computer which allow unconstrained delegation:

```

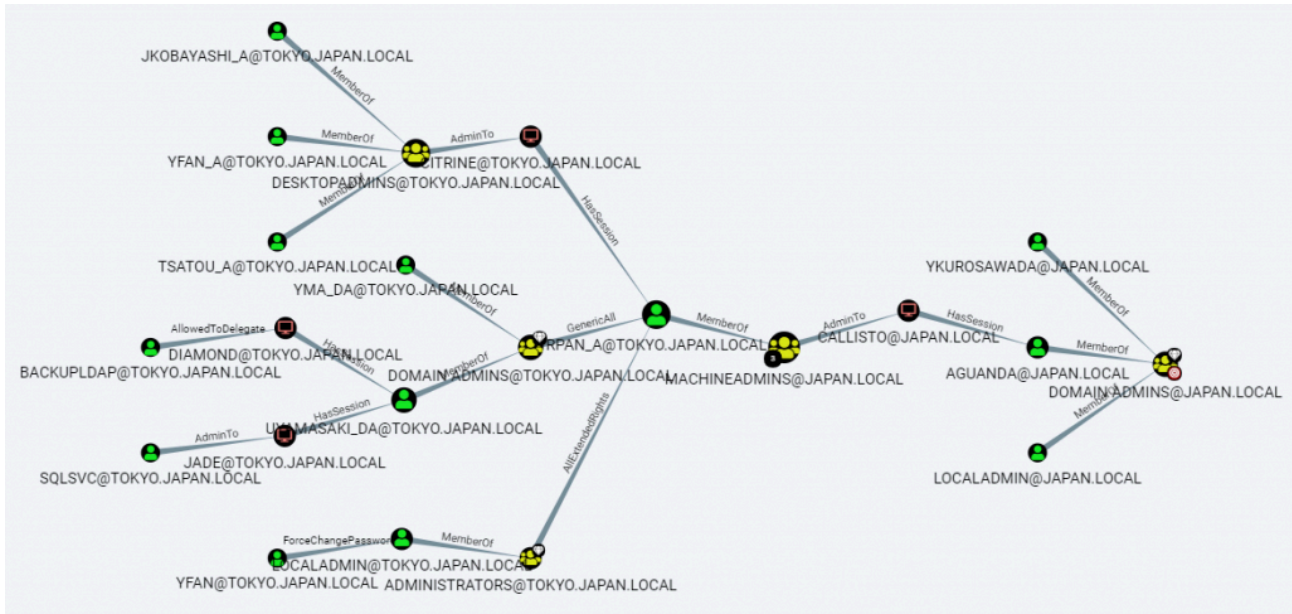
MATCH (dc:Computer)-[:MemberOf*1..]->(g:Group) WHERE g.objectsid ENDS WITH
"516" WITH COLLECT(dc) as domainControllers MATCH p = (d:Domain)-
[:Contains*1..]->(c:Computer {unconstraineddelegation:true}) WHERE NOT c in
domainControllers RETURN p
  
```

5. Shortest path to Domain Admins

MATCH (n:User), (m:Group {name: "DOMAIN ADMINS@DOMAIN.LOCAL"}),

```
p=shortestPath((n)-[*1..]->(m)) RETURN p
```

By combining the information discussed above (and more!) you can find interesting paths in your AD environment. A classical is the shortest path to Domain Admins. This query will show you paths from users to the Domain Admins group via Group Membership, Administration, Session, ACLs, etc.



Several attack paths exist from simple user to Domain Admins.

Conclusion

BloodHound gives a graphical and easy to handle interface to visualize otherwise complex relationships in Active Directory. It empowers attackers and defenders alike with a new point of view on their environments. Some basic queries have been presented here but it is possible to do much more and customize the queries when diving into the Cypher query language.

BloodHound has already been heavily discussed by many sources, good references on the topic are:

- https://www.ernw.de/download/BloodHoundWorkshop/ERNW_DogWhispererHandbook.pdf A great reference document about many BloodHound features
- <https://blog.cptjesus.com/> The blog of one of the authors
- Twitter accounts of the authors: [@_wald0](#), [@harmj0y](#) and [@CptJesus](#)